APPLICATION SPECIFICATIONS

SAFEBASE

Database Backup and Restore Management

B3 Web - La plateforme

Elodie Boweren • Emmanuelle Mellinand-Richier • Armelle Pouzioux

1. Context and Project Presentation

Why this project?

Company data is highly critical and can easily be lost due to technical issues, human error, or database corruption.

SAFEBASE provides a solution to back up and restore databases efficiently, significantly mitigating data loss risks.

Current situation

The objective is to create a unified platform allowing users to:

- Add and manage target databases.
- Schedule automated backups.
- View existing backup archives.
- Restore specific database versions.
- Receive notifications and failure alerts.

Main goals

The project key goals are to:

- Safeguard critical organization data.
- Automate periodic backup routines.
- Simplify and secure restoration operations.
- Maintain structured metadata for each backup event.
- Alert system administrators immediately in case of backup or restoration failure.

2. Project Objectives

Main objectives

The application must allow users to:

- Add MySQL, PostgreSQL, and MongoDB databases.
- Create regular backup schedules.
- Execute automatic backups via cron jobs.
- Maintain versioned backup histories and restore designated versions.
- Monitor operational status across backup and restore processes.
- Receive instant alerts on operation failures.
- Manage all operations via an intuitive web dashboard.

Measurable objectives

By project completion :

- Support MySQL, PostgreSQL, and MongoDB integration.
- Capabilities for both manual and automated backup execution.
- Multi-version storage and management for backup snapshots.
- Seamless restoration capability for selected snapshots.
- Comprehensive error reporting for failed backup or restore actions.
- Thorough test coverage of main core features.
- Full application deployment capability via Docker containerization.

3. Target Users

The primary user of the application is the **Administrator / IT Manager** responsible for database governance.

Key responsibilities and capabilities include:

- Adding and managing database connections.
- Configuring backup schedules.
- Executing manual backups on demand.
- Reviewing backup logs and operational history.
- Restoring database backups safely.
- Monitoring system health, errors, and automated notifications.

The user interface is designed to be clean, intuitive, and efficient for administrators.

Additionally, the layout will fully adapt across various display resolutions and devices.

4. Expected Features

Must Have Features

User Account Management

User access capabilities include:

- Secure authentication and login session management.
- Password protection using strong hashing algorithms (bcrypt, password salted and hashed).

Database Management

Users can execute CRUD operations on database configurations:

- Add new database targets.

- View active database lists and metrics.
- Remove database integrations.
- Select engine types (MySQL, PostgreSQL, MongoDB).

Backup Management

Backup functionality allows administrators to:

- Trigger immediate manual backups.
- Browse global backup archives.
- Inspect specific backup file details.
- Retain multi-version backup history.
- Purge outdated backup files manually or via retention policies.

Recorded metadata for each backup instance (for Postgres and MySQL) includes:

- Target database identifier.
- Timestamp of completion.
- Storage file path.
- Archive file size.

Schedule Management

Automated schedule options allow users to:

- Define the type of backup desired (full, incremental, differential backup)
- Define custom automated backup schedules.
- Select specific target databases per schedule.
- Modify existing recurring schedules.
- Disable or delete obsolete schedules.
- Execute tasks seamlessly in the background using cron services.

Restore Management

Restoration workflows enable users to:

- Browse available backup versions per database.
- Select specific points in time for restoration.
- Initiate restore processes with safety confirmations.
- Track real-time restoration progress and outcomes.

Alerting System

The system proactive monitoring will:

- Automatically detect failed backup and restore jobs.
- Display visual alert indicators within the web interface.
- Provide diagnostic stack traces or details to aid troubleshooting.

Frontend User Interface

The web application includes dedicated administration views for:

- Database Configurations
- Schedules & Automation
- Backup Archives
- Restoration Management
- Alert & Health Monitoring

The interface must be strictly responsive, and aligned with validated UX mockups.

Nice to Have Features

- Advanced log filtering and search capabilities.

5. Project Constraints

Technical Architecture

- Backend powered by Node.js runtime.
- Frontend built using React framework.
- Communication handled via structured REST API endpoints.
- Database support covering MySQL, PostgreSQL, and MongoDB.
- Backup automation powered by cron scheduler services.
- Full environment orchestrated using Docker containerization.

The Docker container setup must encapsulate:

- Backend API services.
- Frontend Web application.
- Databases.

The repository must integrate comprehensive automated unit/integration tests, robust error logging, and standard code linters.

Security Requirements

- User credentials must be salted and hashed (e.g. bcrypt).
- Database passwords and connection strings must be encrypted at rest.
- Endpoints and features are guarded by strict authorization middleware.

Organization & Workflow

The project is developed collaboratively by a team of 3 developers using Agile practices.

Primary organizational toolset includes:

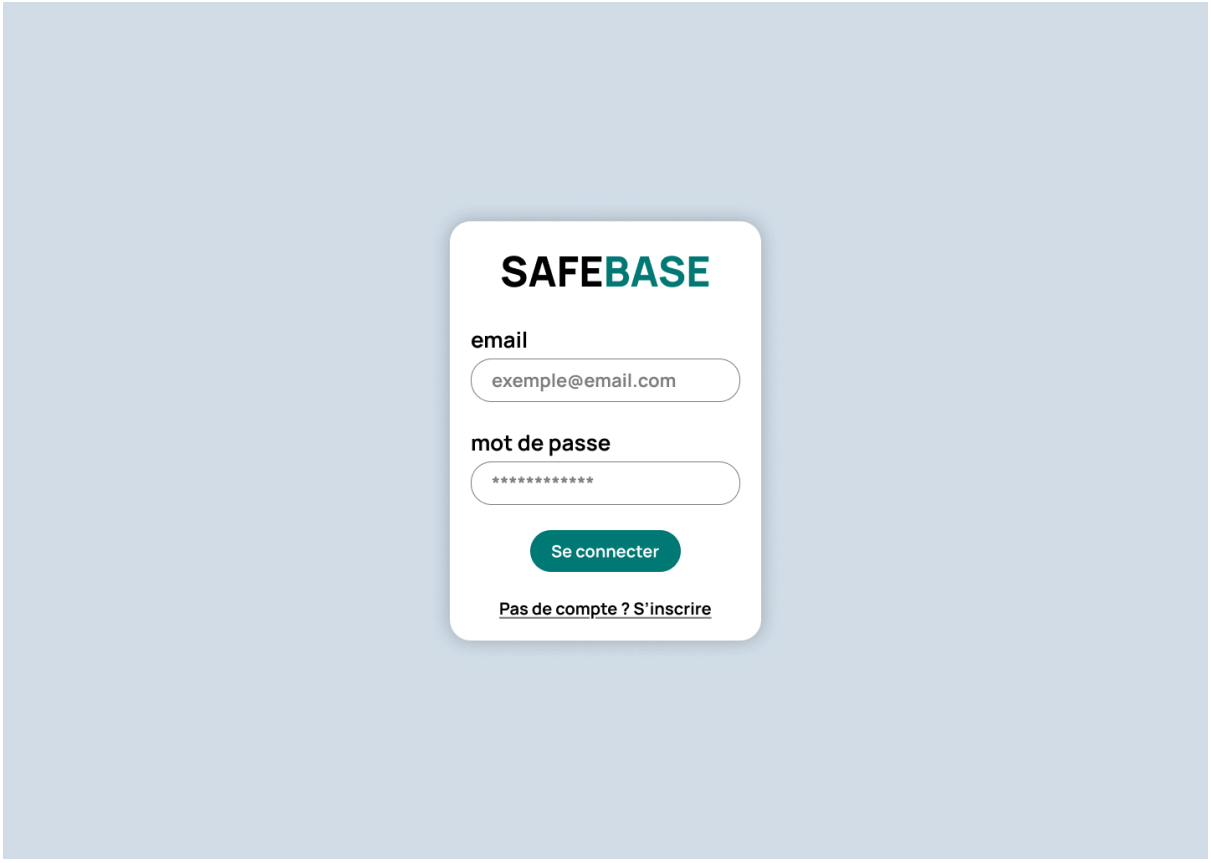
- Git version control & GitHub repository workflow.
- GitHub Projects / Kanban boards for task management.
- Agile methodology.

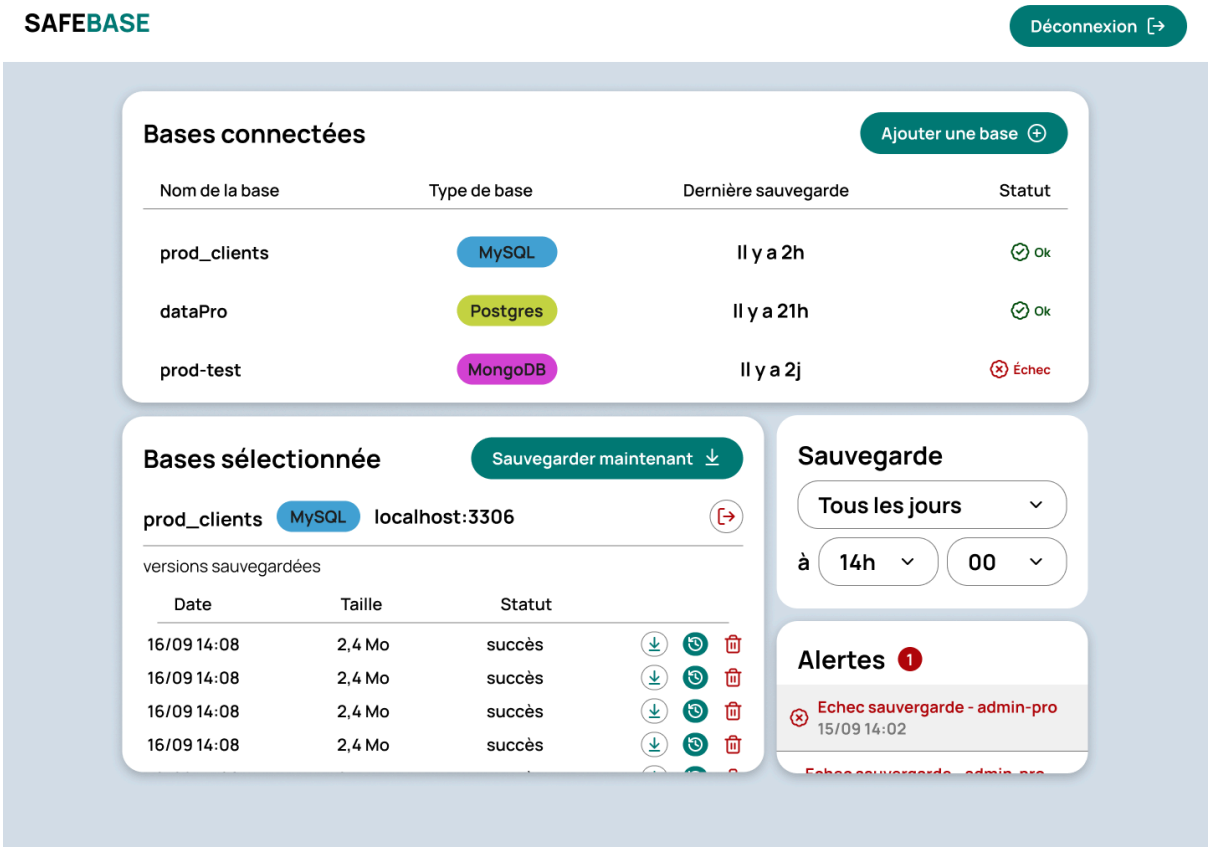
6. Graphic Charter and Design

The user interface will adhere to a modern, minimalist design system to maximize usability and clarity.

Key graphic design requirements include:

- Clear and structured navigation menus.
- Legible typography hierarchy and font sizing.
- Consistent color palettes aligned with SAFEBASE branding.
- Distinct state banners (success, warning, error, info).
- Fully responsive layouts across screens.
- Figma / UI Mockups designed and validated prior to frontend implementation.





7. Expected Deliverables

Intermediate Deliverables (During Project)

- Requirements specification document.
- Backlog & Agile user stories.
- Functional workflow diagrams.
- Interactive UI mockups and prototypes.
- Data models (MCD, MLD, MPD).
- System & software architecture specifications.
- OpenAPI / Swagger documentation.
- Quality assurance & testing strategy document.
- Test execution logs (unit, integration, e2e).
- Agile board tracking links.

Final Deliverables (Project Completion)

- Production-ready React web frontend.
- Fully operational Node.js REST API service.

- Integrated database environment drivers (MySQL, PostgreSQL, MongoDB).
- Automated backup and recovery system engine.
- Cron scheduling automation module.
- Alerting and failure notification service.
- Automated test suites (unit, integration, end-to-end).
- Containerized Docker Compose configuration files.
- Complete technical setup and deployment user documentation.